

MATLAB SESSION 1

Arithmetic, Variables and Arrays

Dr Sumanth Bharadwaj H S
Department of Mathematics and Statistics
M S Ramaiah University of Applied Sciences.

Topics Covered

1. Getting started

- ▶ The MATLAB windows
- ▶ The Command Window
- ▶ Variables and naming rules

2. Arithmetic

- ▶ Operators and precedence
- ▶ Built-in functions
- ▶ Display formats

3. Arrays and matrices

- ▶ MATLAB as a matrix language
- ▶ Creating vectors and matrices
- ▶ Addressing elements

4. Operations on arrays

- ▶ Matrix versus element-wise
- ▶ Functions on whole arrays
- ▶ Solving a linear system

Session outcomes

On completion of this session the student will be able to

1. Evaluate expressions in the Command Window, applying the order of precedence and selecting an appropriate display format.
2. Create vectors and matrices, address any element, row, column or block, and carry out an elementary row operation as a single assignment.
3. Distinguish matrix operations from element-by-element operations, and select the correct one for a given calculation.
4. Solve a linear system using the left division operator, and verify the solution obtained.

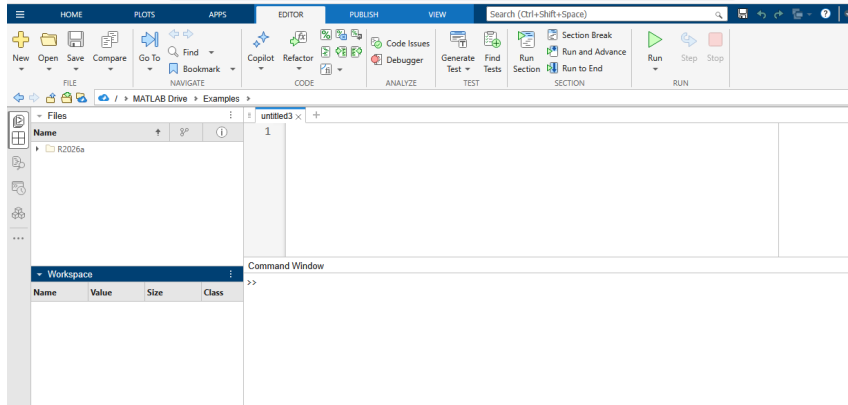
The MATLAB windows

1. Go to `matlab.mathworks.com` and sign in with your MSRUAS email address.
2. Create a folder called MATLAB and keep all your work in it.
3. Files live in MATLAB Drive, so work saved in the lab is on your own machine later.

Window	Purpose
Command Window	Type a command, press Enter, see the result immediately
Workspace	Lists the variables that currently exist
Current Folder	Where MATLAB looks for your files
Command History	Every command you have typed; double-click to run it again
Editor	Where script files are written (Session 2)

The MATLAB windows (continued)

The up-arrow key recalls previous commands. Typing the first few characters and then pressing it recalls only commands starting that way, which saves a great deal of retyping.



MATLAB SESSION 1

MATLAB as a calculator

Typing in the Command Window

Type an expression, press Enter, and MATLAB replies with `ans =` followed by the result.

```
>> 9+6/3  
ans =  
    11
```

Typing in the Command Window (continued)

Symbol	Effect
;	At the end of a command, suppresses display of the result. The command still runs.
,	Separates several commands on one line; results are displayed.
%	Everything after it on the line is a comment and is ignored.
...	Continues the expression on the next line.
clc	Clears the Command Window. Variables are not affected.

Typing in the Command Window (continued)

```
>> 5+3, 5-3  
ans =  
      8  
ans =  
      2  
  
>> 5*3;  
>>
```

Type this now. Type 12/5 and press Enter. Then type 12/5; and notice that nothing is displayed. Then press the up-arrow twice to recall the first command.

The arithmetic operators

Operation	Symbol	Example
Addition	+	5 + 3
Subtraction	-	5 - 3
Multiplication	*	5 * 3
Right division	/	5 / 3
Left division	\	5 \ 3 means 3/5
Exponentiation	^	5 ^ 3 means $5^3 = 125$

Every symbol except the left division is the same as on a calculator. For two numbers the left division reverses the roles, so 5\3 and 3/5 are both 0.6.

The arithmetic operators (continued)

Left division looks pointless for two numbers, and it is. Its purpose appears at the end of this session, where $A \setminus b$ solves a system of equations. It is worth meeting the symbol now so that it is not a surprise then.

Type this now. Confirm that $5 \setminus 3$ and $3/5$ agree. Then evaluate 2^{10} and $2^{0.5}$.

Order of precedence

Precedence	Operation
First	Parentheses. For nested parentheses, the innermost is evaluated first.
Second	Exponentiation.
Third	Multiplication and division, which have equal precedence.
Fourth	Addition and subtraction.

Where two operations have equal precedence, MATLAB works from left to right.
Parentheses override everything, and cost nothing.

Order of precedence (continued)

```
>> 9+6/3  
ans =  
    11  
  
>> (9+6)/3  
ans =  
     5
```

In the first, $6/3$ is evaluated first because division outranks addition. In the second, the parentheses force the addition first.

Type this now. Predict each, then check: $10-6/2*3$, $4^2/8$, $(10-6)/(2*3)$.

A precedence trap

These two differ by one pair of parentheses and give completely different answers.

```
>> 64^(1/3)+16^0.5  
ans =  
      8  
  
>> 64^1/3+16^0.5  
ans =  
25.3333
```

In the first, $1/3$ is evaluated inside the parentheses, so this is the cube root of 64 plus the square root of 16, that is $4 + 4$. In the second, exponentiation outranks division, so 64^1 is evaluated first and then divided by 3, giving $64/3 + 4 = 25.3333$.

When you type a fraction that you wrote as a stacked fraction on paper, the bar was doing the work of a pair of parentheses. Put them back in.

A precedence trap (continued)

Type this now. Evaluate $\frac{3 + 4^2}{\sqrt{5}}$ in one line. The answer is 8.4971. Then evaluate it wrongly by omitting the parentheses and compare.

Long expressions

Three dots continue an expression onto the next line, so a long formula stays readable.

```
>> 1 + 1/factorial(1) + 1/factorial(2) ...  
+ 1/factorial(3) + 1/factorial(4) + 1/factorial(5)  
ans =  
    2.7167
```

That is the first six terms of the series for e , and 2.7167 is already close to the true value 2.7183.

Type this now. Type the expression above across two lines, then add the next two terms and see how much closer to $\exp(1)$ you get.

Display formats

format controls how numbers are shown. It changes nothing about how they are stored or computed.

Command	Description	169/9 displays as
<code>format short</code>	Default, 4 decimal digits	18.7778
<code>format long</code>	15 decimal digits	18.777777777777779
<code>format short e</code>	Scientific, 4 digits	1.8778e+01
<code>format short g</code>	Best of fixed or floating	18.778
<code>format bank</code>	Two decimal digits	18.78
<code>format rat</code>	Nearest ratio of integers	169/9
<code>format compact</code>	Removes blank lines between results	

Display formats (continued)

`format rat` is worth remembering for this course. When a calculation should give an exact fraction, it turns 0.3333 back into $1/3$, which makes comparison with hand working immediate.

Type this now. Evaluate $169/9$, then repeat after `format long`, `format bank` and `format rat`. Return to the default with `format short`.

MATLAB SESSION 1

Built-in functions

Calling a function

A function has a name and an argument in parentheses. The argument can be a number, a variable, an expression, or another function call.

Syntax:

```
name(argument)
```

Calling a function (continued)

```
>> sqrt(144)
ans =
    12

>> sqrt(20+5*13)
ans =
    9.2195

>> sqrt(28+8*sqrt(81))
ans =
    10

>> (40+300/6)/sqrt(100)
ans =
    9
```

Calling a function (continued)

The four lines show the argument as a number, as an expression, as an expression containing another function, and finally the whole function used inside a larger expression.

Type this now. Evaluate `sqrt(11+sqrt(21+sqrt(16)))`. Work it out by hand first; the answer is a whole number.

Elementary mathematical functions

Function	Description	Example
<code>sqrt(x)</code>	Square root	<code>sqrt(81)</code> is 9
<code>nthroot(x,n)</code>	Real nth root	<code>nthroot(80,5)</code> is 2.4022
<code>exp(x)</code>	e^x	<code>exp(5)</code> is 148.4132
<code>abs(x)</code>	Absolute value	<code>abs(-24)</code> is 24
<code>log(x)</code>	Natural logarithm	<code>log(1000)</code> is 6.9078
<code>log10(x)</code>	Base 10 logarithm	<code>log10(1000)</code> is 3
<code>factorial(x)</code>	$x!$	<code>factorial(5)</code> is 120

Elementary mathematical functions (continued)

`log` is the natural logarithm, not base 10.

Type this now. Verify that `log(exp(3))` is 3 and `log10(1e6)` is 6. Then compute `nthroot(-8,3)` and see what `(-8)^(1/3)` gives instead.

Trigonometric functions

Function	Description	Example
<code>sin(x)</code> , <code>cos(x)</code> , <code>tan(x)</code>	Argument in radians	<code>sin(pi/6)</code> is 0.5000
<code>sind(x)</code> , <code>cosd(x)</code> , <code>tand(x)</code>	Argument in degrees	<code>cosd(30)</code> is 0.8660
<code>asin(x)</code> , <code>atan(x)</code>	Inverse, result in radians	<code>asin(1)</code> is 1.5708
<code>asind(x)</code> , <code>atand(x)</code>	Inverse, result in degrees	<code>asind(1)</code> is 90

The plain names take radians. A degree value passed to `sin` gives a plausible wrong answer with no warning: `tan(15)` is -0.8560 , whereas `tand(15)` is 0.2679 .

Type this now. A ramp rises at 15° over a horizontal run of 12 m. Compute the rise with `tand`, then with `tan`, and see how wrong the second is.

Rounding functions

Function	Description	Example
<code>round(x)</code>	To the nearest integer	<code>round(17/5)</code> is 3
<code>fix(x)</code>	Towards zero	<code>fix(13/5)</code> is 2
<code>ceil(x)</code>	Towards $+\infty$	<code>ceil(11/5)</code> is 3
<code>floor(x)</code>	Towards $-\infty$	<code>floor(-9/4)</code> is -3
<code>rem(x,y)</code>	Remainder of x/y	<code>rem(13,5)</code> is 3
<code>sign(x)</code>	1, -1 or 0	<code>sign(5)</code> is 1

`fix` and `floor` agree for positive numbers and differ for negative ones, which is the usual source of confusion between them.

Rounding functions (continued)

Type this now. Evaluate `fix(-3.7)`, `floor(-3.7)`, `ceil(-3.7)` and `round(-3.7)` and account for all four answers.

MATLAB SESSION 1

Variables

The assignment operator

The equals sign assigns: it evaluates the right-hand side and stores the result under the name on the left. Only a single variable name may appear on the left.

```
>> x = 15
x =
    15

>> y = 3*x^2
y =
    675

>> x = x+1
x =
    16
```

The last line is an instruction, not an equation. It reads: take the current value of x , add one, and store the result back in x .

The assignment operator (continued)

Type this now. Set $a = 4$ and $b = 6$, then compute $c = \text{sqrt}(a^2 + b^2)$ without displaying it, and display c on the next line.

Rules about variable names

- ▶ Must begin with a letter, followed by letters, digits or underscores.
- ▶ Up to 63 characters; no spaces, hyphens or punctuation.
- ▶ Case sensitive, so AA, Aa and aa are three different variables.
- ▶ Avoid the names of built-in functions and predefined variables.

Nothing prevents you from writing `sin = 5`. Afterwards `sin(pi/6)` no longer computes a sine; it tries to read an element of your variable and fails. If a familiar function stops working, this is why. Type `clear sin` to recover it.

Type this now. Try `cos = 3`, then `cos(pi)`, and read the error. Recover with `clear cos` and check that `cos(pi)` works again.

Predefined variables

Name	Meaning
ans	The result of the last expression not assigned to a variable.
pi	π .
eps	The smallest distinguishable difference near 1, about 2.2×10^{-16} .
Inf	Infinity, from a division such as $1/0$.
NaN	Not a number, from $0/0$. It spreads through any later calculation.
i, j	The square root of -1 .

All of these can be overwritten, and none of them should be. Assigning to `i` is a common accident, because it is a natural name for a counter.

Predefined variables (continued)

Type this now. Evaluate $1/0$, $0/0$ and $5+\text{NaN}$. Then check the values of `pi` and `eps`.

Commands for managing variables

Command	Effect
<code>who</code>	Lists the variables currently in memory.
<code>whos</code>	Lists them with size, bytes and class.
<code>clear</code>	Removes all variables.
<code>clear x y</code>	Removes only <code>x</code> and <code>y</code> .
<code>clc</code>	Clears the display; variables are untouched.

Type this now. Create three variables, run `whos`, remove one with `clear`, and run `whos` again.

Decimals are stored approximately

Numbers are held in binary. The decimal 0.1 has no exact binary form, in the same way that $1/3$ has no exact decimal form, so it is stored slightly rounded. The double equals sign `==` asks whether two values are exactly equal, and returns 1 for true or 0 for false.

```
>> 0.1+0.2 == 0.3
ans =
    logical
     0

>> format long
>> (0.1+0.2)-0.3
ans =
    5.551115123125783e-17
>> format short
```

Decimals are stored approximately (continued)

The difference is about a quarter of `eps`. Tiny, but not zero, and `==` asks an exact question.

Never test computed decimals with `==`. Test the size of the difference instead:

```
abs((0.1+0.2) - 0.3) < 1e-10
```

Every iterative method you write in this course will stop on a test of that shape.

Type this now. Reproduce the calculation above, then divide the difference by `eps` to see how many units of rounding it represents.

MATLAB SESSION 1

Arrays

The idea the whole language is built on

Why MATLAB is not just a calculator

MATLAB's distinguishing feature is that it is a **matrix language**. Operations on whole arrays are part of the core syntax, not an add-on library. It is not a general-purpose language with a matrix toolbox attached; the matrix is the basic object of the language itself.

The practical consequence is that **everything in MATLAB is an array**. There is no separate kind of thing called a single number. What you would call a scalar is simply an array with one row and one column.

What you call it	Its size	Example
Scalar	1×1	$x = 7$
Row vector	$1 \times n$	$v = [3 \ 1 \ 4]$ is 1×3
Column vector	$m \times 1$	$b = [8; -11; -3]$ is 3×1
Matrix	$m \times n$	$A = [1 \ 2 \ 3; 4 \ 5 \ 6]$ is 2×3

Why MATLAB is not just a calculator (continued)

So `size` works on absolutely anything, including a plain number.

```
>> x = 7;  
>> size(x)  
ans =  
     1     1
```

Type this now. Check `size` on a number, on `[3 1 4]`, and on `[3; 1; 4]`. Confirm the last two are different sizes even though they hold the same three values.

Advantage of matrix language

Because array operations are built into the language, a calculation that would need a loop elsewhere is written as a single expression here.

In a general-purpose language, in outline:

```
for each element k of x:  
    y[k] = square root of x[k]
```

You describe the repetition yourself, one element at a time.

Both give the same answer. The second is shorter, but more importantly it is the way the language is meant to be used, and it is how nearly every command in these sessions will be written.

In MATLAB

```
y = sqrt(x);
```

You state what you want of the whole array, and the repetition is implied.

Whenever you find yourself about to write a loop over the elements of an array, pause and ask whether one array expression would do it instead. Very often it will.

Advantage of matrix language

Type this now. Let $x = [0 \ 1 \ 4 \ 9 \ 16]$. Compute the square root of every element with a single command, with no loop.

Creating a vector

A vector is written in square brackets, with spaces or commas between the elements.

```
>> v = [3 1 4 1 5]
v =
     3     1     4     1     5
```

Form	Meaning
m:q:n	From m in steps of q, stopping at or before n
m:n	The same with a step of 1
linspace(a,b,n)	n values evenly spaced from a to b, both included

Creating a vector (continued)

```
>> x = 1:2:9
x =
     1     3     5     7     9

>> y = linspace(0,1,5)
y =
     0    0.2500    0.5000    0.7500    1.0000
```

With the colon form you choose the step, and the final value may not be reached: `0:0.3:1` stops at 0.9. With `linspace` you choose how many points, and both endpoints are always included.

Type this now. Produce the even numbers from 2 to 20 with the colon form, and eleven values from 0 to 1 with `linspace`.

Creating a matrix

A semicolon inside the brackets starts a new row. Every row must have the same number of elements.

```
>> A = [1 2 3; 4 5 6]
A =
     1     2     3
     4     5     6
```

Command	Result
<code>zeros(m,n)</code>	An $m \times n$ matrix of zeros
<code>ones(m,n)</code>	An $m \times n$ matrix of ones
<code>eye(n)</code>	The $n \times n$ identity matrix
<code>rand(m,n)</code>	Random values between 0 and 1

Creating a matrix (continued)

A single argument means square. `zeros(3)` is 3×3 , not 3×1 . For one row write `zeros(1,3)`.

Type this now. Create the 3×3 identity, a 2×4 matrix of ones, and a 3×3 matrix in which every entry is 7.

The transpose operator

A single quote after a variable interchanges its rows and columns.

```
>> v = [3 1 4]
v =
     3     1     4

>> u = v'
u =
     3
     1
     4
```

This is how a row vector becomes a column vector, which matters when a system of equations needs its right-hand side as a column.

Type this now. Create `A = [1 2 3; 4 5 6]` and display `A'`. What size is the result?

Addressing elements

Elements are numbered from 1. A vector takes one index; a matrix takes a row and a column.

Syntax:

<code>v(index)</code>	<code>% one element of a vector</code>
<code>A(row, column)</code>	<code>% one element of a matrix</code>

For example:

```
>> v = [3 1 4 1 5];  
>> v(3)  
ans =  
    4  
  
>> A = [1 2 3; 4 5 6];  
>> A(2,3)  
ans =  
    6
```

An element can also be assigned to, which changes the array in place.

Addressing elements (continued)

```
>> v(2) = 99
```

```
v =
```

```
    3    99    4    1    5
```

Indices must be positive whole numbers. `v(0)` and `v(2.5)` are errors, and reading beyond the end of an array is an error, though assigning beyond it is not.

Type this now. With `A = [1 2 3; 4 5 6]`, display the element in row 1 column 2, then change it to 10.

Using a colon in addressing

A colon on its own means every value along that dimension.

<code>A(2,:)</code>	All of row 2
<code>A(:,3)</code>	All of column 3
<code>A(1:2, 2:3)</code>	Rows 1 to 2, columns 2 to 3
<code>v(2:4)</code>	Elements 2 to 4 of a vector
<code>A(end,:)</code>	The last row, whatever the size of A
<code>A(:)</code>	Every element, as a single column

Using a colon in addressing (continued)

```
>> A = [1 2 3; 4 5 6; 7 8 9];
```

```
>> A(2,:)
```

```
ans =
```

```
    4    5    6
```

```
>> A(:, [1 3])
```

```
ans =
```

```
    1    3  
    4    6  
    7    9
```

The second example shows that an index may itself be a vector, here selecting columns 1 and 3.

end means the last index in that dimension. Prefer it to a fixed number, so that a command still works if the array changes size.

Using a colon in addressing (continued)

Type this now. From `A = magic(4)`, extract the third row, the last column, and the 2×2 block in the bottom right.

Adding and deleting elements

Assigning beyond the end of an array extends it, filling any gap with zeros.

```
>> v = [1 2 3];  
>> v(6) = 9  
v =  
     1     2     3     0     0     9
```

A whole row or column can be replaced, or removed using empty brackets.

```
>> A = [1 2 3; 4 5 6; 7 8 9];  
>> A(2,:) = [10 11 12];  
>> A(3,:) = []  
A =  
     1     2     3  
    10    11    12
```

Adding and deleting elements (continued)

Only whole rows or whole columns may be deleted, because what remains must still be rectangular. $A(2,3) = []$ is an error.

Type this now. Build a 3×3 matrix, replace its second row with zeros, then delete its first column.

Functions for handling arrays

Function	Result
<code>size(A)</code>	A row containing the number of rows and columns
<code>numel(A)</code>	The total number of elements
<code>length(v)</code>	The length of a vector
<code>reshape(A,m,n)</code>	The same elements arranged as $m \times n$
<code>diag(A)</code>	The diagonal of a matrix, as a column
<code>sort(v)</code>	The elements in ascending order
<code>max(v), min(v)</code>	Largest and smallest element
<code>sum(v), mean(v)</code>	Total and average

Functions for handling arrays (continued)

Applied to a matrix, `sum`, `max` and `mean` work down each column and return a row of answers. `sum(A(:))` totals the whole matrix.

Type this now. With `A = magic(4)`, check that every column has the same total using `sum(A)`, and find the largest element with `max(max(A))`.

MATLAB SESSION 1

Operations with arrays

Addition, subtraction and scalars

Two arrays of the same size are added and subtracted element by element. A scalar applies to every element.

```
>> A = [1 2; 3 4]; B = [5 6; 7 8];  
>> A+B  
ans =  
     6     8  
    10    12  
  
>> 2*A  
ans =  
     2     4  
     6     8
```

Adding arrays of different sizes fails in the cases you will meet here: $[1 \ 2 \ 3] + [1 \ 2]$ is an error, and the message names the sizes involved.

Addition, subtraction and scalars (continued)

Type this now. Create two 2×3 matrices and compute their sum, their difference, and three times the first.

Matrix multiplication

$A*B$ is the matrix product. The number of columns of A must equal the number of rows of B .

```
>> A = [1 2; 3 4]; B = [5 6; 7 8];  
>> A*B  
ans =  
    19    22  
    43    50
```

The entry 19 is the first row of A against the first column of B , that is $1 \times 5 + 2 \times 7$. Order matters: $A*B$ and $B*A$ are generally different matrices.

Type this now. Compute $A*B$ and $B*A$ for the matrices above and confirm they differ. Then verify that $A*\text{eye}(2)$ returns A .

Element-by-element operations

Placing a dot before $*$, $/$ or $^$ makes the operation act on corresponding elements instead.

Operation	Matrix form	Element-by-element
Multiplication	$A*B$	$A.*B$
Division	A/B	$A./B$
Exponentiation	A^2	$A.^2$

Element-by-element operations (continued)

```
>> A.*B
ans =
     5     12
    21     32

>> A.^2
ans =
     1     4
     9    16
```

$A.^2$ squares each entry. A^2 means $A*A$, which is a different matrix entirely.

Use the matrix form when the array represents a system or a transformation. Use the dot form when the array holds a list of values, which is the case whenever you evaluate a formula at many points at once.

Element-by-element operations (continued)

Type this now. With `x = 1:5`, compute `x.^2` and `x.*x` and confirm they agree. Then try `x^2` and read the error message.

Arrays inside built-in functions

A mathematical function applied to an array acts on every element, so no repetition is needed.

```
>> x = [0 1 4 9 16];  
>> sqrt(x)  
ans =  
     0     1     2     3     4  
  
>> A = [1 2; 3 4];  
>> exp(A)  
ans =  
    2.7183    7.3891  
   20.0855   54.5982
```

This is why MATLAB code is usually shorter than the equivalent elsewhere, and it is used constantly in Session 2 when evaluating a function for plotting.

Arrays inside built-in functions (continued)

Type this now. Let $t = 0:0.25:1$. Compute $\sin(\pi*t)$ in one command, and check the values at $t = 0, 0.5$ and 1 by hand.

Solving a system of equations

For a system written as $Ax = b$ with A square, the left division operator produces the solution.

Syntax:

```
x = A\b           % solves A*x = b
```

For example:

```
>> A = [2 1 -1; -3 -1 2; -2 1 2];  
>> b = [8; -11; -3];  
>> x = A\b  
x =  
     2  
     3  
    -1
```

Note that b is entered as a column, using semicolons. The answer is also a column.

Solving a system of equations (continued)

Function	Result
<code>inv(A)</code>	The inverse, when it exists
<code>det(A)</code>	The determinant
<code>rank(A)</code>	The rank
<code>A'</code>	The transpose

`A\b` is preferred to `inv(A)*b`. It is faster and numerically better behaved, because it never forms the inverse. Reserve `inv` for when you genuinely want the inverse matrix itself.

Type this now. Solve the system above, then check by computing $A*x - b$, which should be zero apart from rounding. Also compute `det(A)` and `rank(A)`.

Errors you will meet

Message	Usual cause
Unrecognized function or variable	A typo, or a variable that was cleared
Index exceeds the number of array elements	Asking for an element that is not there
Matrix dimensions must agree	Element-by-element operation on different sizes
Incorrect dimensions for raising a matrix to a power	x^2 where $x.^2$ was meant

When something fails, read the whole message, then check the sizes of the variables involved with `size` before changing anything.

Type this now. Deliberately produce each of the four errors above, so that you recognise them when they happen by accident.

What you should be able to do now

- ▶ Use the Command Window as a calculator, and control the order of operations with parentheses.
- ▶ Suppress output with a semicolon and continue a long expression with three dots.
- ▶ Change the display with `format`, including `format rat` for exact fractions.
- ▶ Call built-in functions, and choose between radians and degrees.
- ▶ Create variables, follow the naming rules, and manage them with `who` and `clear`.
- ▶ Explain why `0.1+0.2 == 0.3` is false, and compare decimals correctly.
- ▶ Create vectors with the colon and `linspace`, and matrices with brackets, `zeros`, `ones` and `eye`.
- ▶ Address any element, row, column or block, and use `end`.
- ▶ Add and delete rows and columns.
- ▶ Choose correctly between `*` and `.*`, and between `^` and `.^`.
- ▶ Solve a system with `A\b`, and check the result.

Problems

Work through these before Session 2.

1. Evaluate each by hand first, then check in MATLAB. Where you differ, identify which precedence rule you overlooked.

- | | |
|---------------------------|-------------------------|
| (a) $6+4/2*3$ | (b) $(6+4)/(2*3)$ |
| (c) 3^2*2 | (d) $3^(2^2)$ |
| (e) $16^{0.5}+27^{(1/3)}$ | (f) $16^{0.5}+27^{1/3}$ |

2. Compute each, giving the answer to four decimal places.

- ▶ $\frac{5 + 3^2}{7 - 2}$
- ▶ $\sqrt{3^2 + 4^2}$
- ▶ $e^{-0.5} \cos(\pi/3)$
- ▶ $\log_{10} 2500$

3. A ramp rises at 15° over a horizontal run of 12 m. Compute the rise using `tand`, then using `tan`. State both answers and explain why the second is negative.

Problems (continued)

4. Create $v = [4 \ -2 \ 7 \ -1 \ 9 \ 3]$. Write one command for each part.

- ▶ The third element
- ▶ Elements 2 to 4
- ▶ The last element, using `end`
- ▶ The vector with its second element changed to 100
- ▶ The elements in ascending order

Problems (continued)

5. Create the matrix below, then write one command for each part.

```
A = [2 5 8 11; 3 6 9 12; 4 7 10 13];
```

- ▶ The element in row 2, column 3
- ▶ All of row 3
- ▶ Columns 2 and 4
- ▶ The 2×2 block in the top left
- ▶ The matrix with row 2 deleted

6. With $A = [1 \ 2; 3 \ 4]$ and $B = [5 \ 6; 7 \ 8]$, compute $A*B$, $A.*B$, A^2 and $A.^2$.

For each, state in one sentence what was computed. 7. Let

$x = \text{linspace}(0, 2, 11)$. Compute $y = x^2 e^{-x}$ as an array. State which operators need a dot, and why.

Problems (continued)

8. Solve the system with left division, then verify by computing $A*x - b$. Also report $\det(A)$ and $\text{rank}(A)$.

```
A = [4 -2 1; -2 4 -2; 1 -2 4];  
b = [11; -16; 17];
```

9. The matrix below is singular. Report what `det`, `rank` and `inv` each give, and what happens when you attempt to solve $S \backslash [1; 2; 3]$.

```
S = [1 2 3; 2 4 6; 1 1 1];
```

10. Set `format rat` and evaluate $1/3 + 1/6$. Then set `format long` and evaluate it again. Explain what each display tells you about the same stored number.